

ARIN5204: Reinforcement Learning

1.4 Planning with Dynamic Programming

Dr. Long Chen

longchen@ust.hk

Assistant Professor, Dept. of CSE
The Hong Kong University of Science and Technology (HKUST)

March 2, 2026



- ① Policy Evaluation
- ② Policy Improvement
- ③ Policy Iteration and Value Iteration
- ④ Value Function in Theory

Some slides are from: David Silver (DeepMind), Sergey Levine (UCB), Emma Brunskill (Stanford), Katerina Fragkiadaki (CMU), Diana Borsa (DeepMind).

① Policy Evaluation

② Policy Improvement

③ Policy Iteration and Value Iteration

④ Value Function in Theory

Iterative Policy Evaluation for MDP

- **Problem:** evaluate a given policy π
- The only thing we know:

$$V_{\pi}(s) = \mathbb{E}[r_t + \gamma V_{\pi}(s_{t+1})]$$

- turn this equality into an update
- **Solution:** iterative application of Bellman expectation backup

$$V_1 \rightarrow V_2 \rightarrow \dots \rightarrow V_{\pi}$$

Policy Evaluation

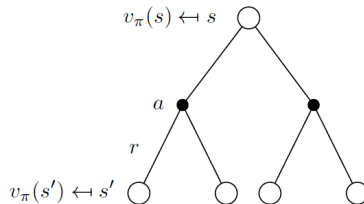
Algorithm

- First, initialize, V_0 , e.g., to zero
- Then, iterate

$$\forall s: V_{k+1}(s) \leftarrow \mathbb{E}[r_t + \gamma V_k(s_{t+1}) | s, \pi]$$

- **Stopping:** whenever $V_{k+1}(s) = V_k(s)$, for all s , we must have found V_π
- **Q: Does this algorithm always converge?**
- **A:** Yes, under appropriate conditions (e.g., $\gamma < 1$). More in the future.

Backup Diagram: Iterative Policy Evaluation for MDP



- In practice: using **synchronous** backups
 - At each iteration $k+1$, for all states $s \in \mathcal{S}$
 - Update $V_{k+1}(s)$ from $V_k(s')$, where s' is a successor state of s

$$V_{k+1}(s) = \sum_{a \in A} \pi(a|s) \left(r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) V_k(s') \right)$$

Algorithm: Iterative Policy Evaluation (in-place)

Iterative Policy Evaluation, for estimating $V \approx v_\pi$

Input π , the policy to be evaluated

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$ arbitrarily, for $s \in \mathcal{S}$, and $V(\text{terminal})$ to 0

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

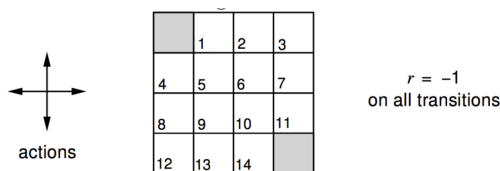
$v \leftarrow V(s)$

$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$

Example: Random Policy in Gridworld (S. & B. Ch4, Page 76)



- Undiscounted episodic MDP ($\gamma = 1$)
- Nonterminal states 1,...,14, two terminal states (shown as shaded squares)
- Actions leading out of the grid leave state **unchanged**
- Reward is 1 until the terminal state is reached
- Agent follows a uniform random policy

$$\pi(N|\cdot) = \pi(E|\cdot) = \pi(S|\cdot) = \pi(W|\cdot) = 0.25$$

Example: Random Policy in Gridworld (S. & B. Ch4, Page 76)

Policy Evaluation

v_k for the
Random Policy

$k = 0$

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$k = 2$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

Example: Random Policy in Gridworld (S. & B. Ch4, Page 76)

Policy Evaluation

 $k = 3$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0

 $k = 10$

0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0

 $k = \infty$

0.0	-14.	-20.	-22.
-14.	-18.	-20.	-20.
-20.	-20.	-18.	-14.
-22.	-20.	-14.	0.0

- ① Policy Evaluation
- ② Policy Improvement
- ③ Policy Iteration and Value Iteration
- ④ Value Function in Theory

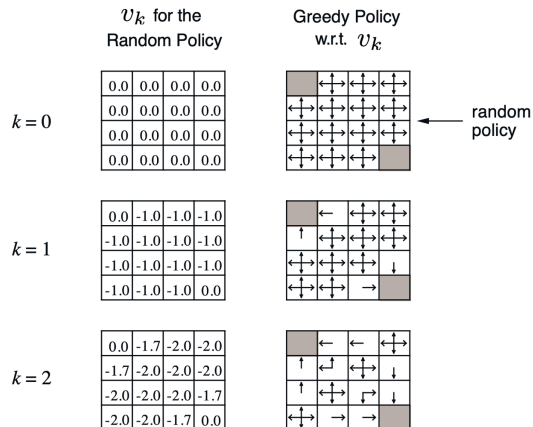
Policy Improvement

- **Q: How to improve a policy?**
- For a reference policy π , we need to find another π' , and it can improve the value function, *i.e.*, $V_{\pi'}(s) \geq V_{\pi}(s)$
- **Solution:** Consider a deterministic policy, $a = \pi(s)$
- We can **improve** the policy by acting greedily

$$\begin{aligned}\pi'(s) &= \arg\max_{a \in A} Q_{\pi}(s, a) \\ &= \arg\max_{a \in A} \mathbb{E}[r(s, a) + \gamma V_{\pi}(s_{t+1}) | s_t = s, a_t = a]\end{aligned}$$

Example: Random Policy in Gridworld (S. & B. Ch4, Page 76)

Policy Evaluation + Greedy Improvements



Policy Evaluation + Greedy Improvements

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0

0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0

optimal
policy

0.0	-14.	-20.	-22
-14.	-18.	-20.	-20
-20.	-20.	-18.	-14
-22.	-20.	-14.	0.0

Policy Improvement Theorem: $V_{\pi'}(s) \geq V_{\pi}(s)$

$$\begin{aligned}\pi'(s) &= \arg\max_{a \in A} Q_{\pi}(s, a), \forall s \in \mathcal{S} \\ \Rightarrow Q_{\pi}(s, \pi'(s)) &\geq V_{\pi}(s)\end{aligned}$$

Proof:

$$\begin{aligned}V_{\pi}(s) &\leq Q_{\pi}(s, \pi'(s)) = \mathbb{E}[r_t + \gamma V_{\pi}(s_{t+1}) | s_t = s, a_t = \pi'(s)] \\ &= \mathbb{E}_{\pi'}[r_t + \gamma V_{\pi}(s_{t+1}) | s_t = s] \\ &\leq \mathbb{E}_{\pi'}[r_t + \gamma Q_{\pi}(s_{t+1}, \pi'(s_{t+1})) | s_t = s] \\ &= \mathbb{E}_{\pi'}[r_t + \gamma \mathbb{E}[r_{t+1} + \gamma V_{\pi}(s_{t+2}) | s_{t+1}, a_{t+1} = \pi'(s_{t+1})] | s_t = s] \\ &= \mathbb{E}_{\pi'}[r_t + \gamma r_{t+1} + \gamma^2 V_{\pi}(s_{t+2}) | s_t = s] \\ &\leq \mathbb{E}_{\pi'}[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 V_{\pi}(s_{t+3}) | s_t = s] \\ &\leq \mathbb{E}_{\pi'}[r_t + \gamma r_{t+1} + \dots | s_t = s] = V_{\pi'}(s)\end{aligned}$$

Policy Improvement Theorem: $Q_{\pi'}(s, a) \geq Q_{\pi}(s, a)$

Likewise

$$\begin{aligned} Q_{\pi}(s, a) &\leq Q_{\pi}(s, \pi'(s)) \\ &= \mathbb{E}_{\pi}[r_t + \gamma Q_{\pi}(s_{t+1}, a_{t+1}) | s_t = s, a_t = \pi'(s)] \\ &\leq \mathbb{E}_{\pi'}[r_t + \gamma Q_{\pi}(s_{t+1}, \pi'(s_{t+1})) | s_t = s, a_t = \pi'(s)] \\ &= \mathbb{E}_{\pi'}[r_t + \gamma \mathbb{E}[r_{t+1} + \gamma Q_{\pi}(s_{t+2}, a_{t+2}) | s_{t+1}, a_{t+1} = \pi'(s_{t+1})] | s_t = s, a_t = \pi'(s)] \\ &\leq \mathbb{E}_{\pi'}[r_t + \gamma r_{t+1} + \gamma^2 Q_{\pi}(s_{t+2}, \pi'(s_{t+2})) | s_t = s, a_t = \pi'(s)] \\ &\leq \mathbb{E}_{\pi'}[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots | s_t = s, a_t = \pi'(s)] \\ &= Q_{\pi'}(s, a_t = \pi'(s)) \end{aligned}$$

How to Improve a Policy

Given a policy π

- **Evaluate** the policy π

$$V_{\pi}(s) = \mathbb{E}[r_t + \gamma r_{t+1} + \dots | s_t = s]$$

- **Improve** the policy by acting greedily with respect to V_{π}

$$\pi' = \text{greedy}(V_{\pi})$$

A Live Demo on Policy Evaluation + Improvement

`https:
//cs.stanford.edu/people/karpathy/reinforcejs/gridworld_dp.html`

- 1 Policy Evaluation
- 2 Policy Improvement
- 3 Policy Iteration and Value Iteration**
- 4 Value Function in Theory

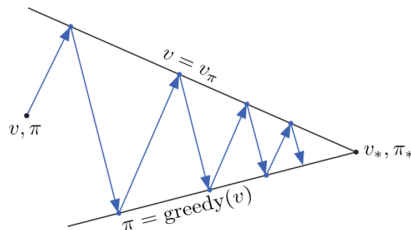
Policy Iteration

Once a policy, π_0 , has been improved using V_{π_0} to yield a better policy, π_1 , we can then compute V_{π_1} and improve it again to yield an even better π_2 .

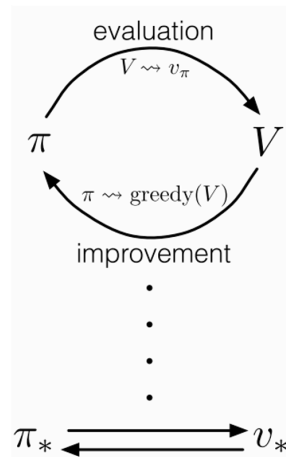
$\xrightarrow{E}$: policy evaluation $\xrightarrow{I}$: policy improvement

$$\pi_0 \xrightarrow{E} V_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} V_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} V_{\pi_2} \xrightarrow{I} \dots \xrightarrow{I} V_*$$

Policy Iteration



- **Policy evaluation** estimate V_π
 - Iterative policy evaluation
- **Policy improvement** generate $\pi' \geq \pi$
 - Greedy policy improvement



Algorithm: Policy Iteration

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$; $V(\text{terminal}) \doteq 0$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

policy-stable $\leftarrow$ true

For each $s \in \mathcal{S}$:

old-action $\leftarrow \pi(s)$

$\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ false

If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Modified Policy Iteration

- Q: Does policy evaluation need to converge to V_π ?
- Or should we introduce a stopping condition
- Or simple stop after k iterations of iterative policy evaluation?
- For example, in the small gridworld $k = 3$ was sufficient to achieve an optimal policy
- Why not update policy every iteration? *i.e.*, stop after $k = 1$
 - Value iteration

Algorithm: Value Iteration

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

| $\Delta \leftarrow 0$

| Loop for each $s \in \mathcal{S}$:

| $v \leftarrow V(s)$

| $V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$

| $\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that

$$\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$$

Value iteration directly takes [the Bellman optimality equation](#), and turns that into an update

Example: Shortest Path

<table> <tr><td>g</td><td></td><td></td><td></td></tr> <tr><td></td><td></td><td></td><td></td></tr> <tr><td></td><td></td><td></td><td></td></tr> <tr><td></td><td></td><td></td><td></td></tr> </table> Problem	g																<table> <tr><td>0</td><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>0</td><td>0</td><td>0</td></tr> </table> V_1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	<table> <tr><td>0</td><td>-1</td><td>-1</td><td>-1</td></tr> <tr><td>-1</td><td>-1</td><td>-1</td><td>-1</td></tr> <tr><td>-1</td><td>-1</td><td>-1</td><td>-1</td></tr> <tr><td>-1</td><td>-1</td><td>-1</td><td>-1</td></tr> </table> V_2	0	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	<table> <tr><td>0</td><td>-1</td><td>-2</td><td>-2</td></tr> <tr><td>-1</td><td>-2</td><td>-2</td><td>-2</td></tr> <tr><td>-2</td><td>-2</td><td>-2</td><td>-2</td></tr> <tr><td>-2</td><td>-2</td><td>-2</td><td>-2</td></tr> </table> V_3	0	-1	-2	-2	-1	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2	-2
g																																																																			
0	0	0	0																																																																
0	0	0	0																																																																
0	0	0	0																																																																
0	0	0	0																																																																
0	-1	-1	-1																																																																
-1	-1	-1	-1																																																																
-1	-1	-1	-1																																																																
-1	-1	-1	-1																																																																
0	-1	-2	-2																																																																
-1	-2	-2	-2																																																																
-2	-2	-2	-2																																																																
-2	-2	-2	-2																																																																
<table> <tr><td>0</td><td>-1</td><td>-2</td><td>-3</td></tr> <tr><td>-1</td><td>-2</td><td>-3</td><td>-3</td></tr> <tr><td>-2</td><td>-3</td><td>-3</td><td>-3</td></tr> <tr><td>-3</td><td>-3</td><td>-3</td><td>-3</td></tr> </table> V_4	0	-1	-2	-3	-1	-2	-3	-3	-2	-3	-3	-3	-3	-3	-3	-3	<table> <tr><td>0</td><td>-1</td><td>-2</td><td>-3</td></tr> <tr><td>-1</td><td>-2</td><td>-3</td><td>-4</td></tr> <tr><td>-2</td><td>-3</td><td>-4</td><td>-4</td></tr> <tr><td>-3</td><td>-4</td><td>-4</td><td>-4</td></tr> </table> V_5	0	-1	-2	-3	-1	-2	-3	-4	-2	-3	-4	-4	-3	-4	-4	-4	<table> <tr><td>0</td><td>-1</td><td>-2</td><td>-3</td></tr> <tr><td>-1</td><td>-2</td><td>-3</td><td>-4</td></tr> <tr><td>-2</td><td>-3</td><td>-4</td><td>-5</td></tr> <tr><td>-3</td><td>-4</td><td>-5</td><td>-5</td></tr> </table> V_6	0	-1	-2	-3	-1	-2	-3	-4	-2	-3	-4	-5	-3	-4	-5	-5	<table> <tr><td>0</td><td>-1</td><td>-2</td><td>-3</td></tr> <tr><td>-1</td><td>-2</td><td>-3</td><td>-4</td></tr> <tr><td>-2</td><td>-3</td><td>-4</td><td>-5</td></tr> <tr><td>-3</td><td>-4</td><td>-5</td><td>-6</td></tr> </table> V_7	0	-1	-2	-3	-1	-2	-3	-4	-2	-3	-4	-5	-3	-4	-5	-6
0	-1	-2	-3																																																																
-1	-2	-3	-3																																																																
-2	-3	-3	-3																																																																
-3	-3	-3	-3																																																																
0	-1	-2	-3																																																																
-1	-2	-3	-4																																																																
-2	-3	-4	-4																																																																
-3	-4	-4	-4																																																																
0	-1	-2	-3																																																																
-1	-2	-3	-4																																																																
-2	-3	-4	-5																																																																
-3	-4	-5	-5																																																																
0	-1	-2	-3																																																																
-1	-2	-3	-4																																																																
-2	-3	-4	-5																																																																
-3	-4	-5	-6																																																																

After the optimal values are reached, run policy extraction to retrieve the optimal policy.

Summary: Policy Iteration vs. Value Iteration

Policy Iteration

1. Evaluation $V_\pi(s)$ # policy evaluation

2. Set $\pi \leftarrow \pi'$

(LOOP) GO TO STEP1

$$\pi'(a_t|s_t) = \begin{cases} 1, & \text{if } a_t = \arg\max_{a_t} Q_\pi(s_t, a_t) \\ 0, & \text{otherwise} \end{cases}$$

Policy Evaluation

$$V_\pi(s) \leftarrow r(s, \pi(s)) + \gamma \mathbb{E}_{s' \sim p(s'|s, \pi(s))} [V_\pi(s')]$$

(LOOP)

Summary: Policy Iteration vs. Value Iteration

Policy Iteration

1. Evaluation $V_{\pi}(s)$ # policy evaluation
(LOOP inside)
 2. Set $\pi \leftarrow \pi'$
- (LOOP) GO TO STEP1

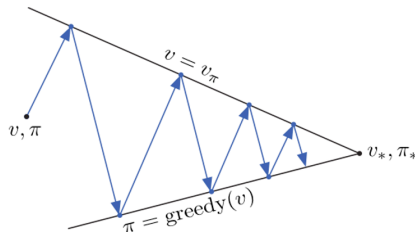
Value Iteration

1. Set $Q(s, a) \leftarrow r(s, \pi(s)) + \gamma \mathbb{E}_{s' \sim p(s'|s, \pi(s))} [V_{\pi}(s')]$
 2. Set $V(s) \leftarrow \max_a Q(s, a)$
- (LOOP) GO TO STEP1

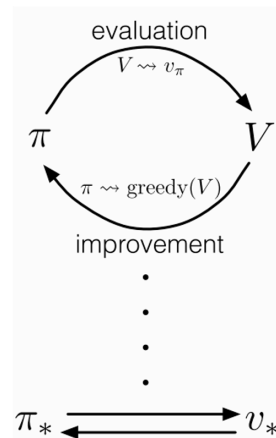
Difference between Policy Iteration and Value Iteration

- **Policy Iteration** includes: **policy evaluation** + **policy improvement**, and the two are repeated iteratively until policy converges.
- **Value Iteration** includes: **finding the optimal value function** + **one policy extraction**. There is no repetition of the two because once the value function is optimal, then the policy out of it should also be optimal (*i.e.*, converged)

Generalized Policy Iteration



- **Policy evaluation** estimate V_π
 - Any policy evaluation algorithm
- **Policy improvement** generate $\pi' \geq \pi$
 - Any policy improvement algorithm



Efficiency of DP

- Algorithms are based on state-value function $V_{\pi}(s)$ or $V_*(s)$
 - Complexity $O(mn^2)$ per iteration, for m actions and n states
- Could also apply to action-value function $Q_{\pi}(s, a)$ or $Q_*(s, a)$
 - Complexity $O(m^2n^2)$ per iteration
- To find an optimal policy is polynomial in the number of states
- But, the number of states is often astronomical, e.g., often growing exponentially with the number of state variables (“curse of dimensionality”)
- In practice, classical DP can be applied to problems with a few million states

Asynchronous Dynamic Programming

- All DP methods described so far require exhaustive sweeps of the entire state set.
- Asynchronous DP does not use sweeps. Instead, it works like this:
 - Repeat until convergence criterion is met:
 - Sample a state at random and apply the appropriate backup
- Still needs lots of computation, but does not get locked into hopelessly long sweeps
- Guaranteed to converge if all states continue to be selected.

Asynchronous Dynamic Programming

Several simple ideas for asynchronous dynamic programming:

- In-place dynamic programming
- Prioritized sweeping
- Real-time dynamic programming

In-Place Dynamic Programming

- **Synchronous** value iteration stores two copies of the value function
 - For all s in $\mathcal{S}$

$$V_{new}(s) \leftarrow \max_a \left(r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) V_{old}(s') \right)$$

$$V_{old}(\cdot) \leftarrow V_{new}(\cdot)$$

- **In-place** value iteration only stores one copy of the value function
 - For all s in $\mathcal{S}$

$$V(s) \leftarrow \max_a \left(r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) V(s') \right)$$

Prioritized Sweeping

- Use the **magnitude of Bellman error** to guide state selection, e.g.,

$$\left| \max_a \left(r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) V(s') \right) - V(s) \right|$$

- Backup the state with the largest remaining Bellman error
- Update Bellman of affected states after each backup
- **Requires knowledge of reverse dynamics (predecessor states)**
- Can be implemented efficiently by maintaining a priority queue

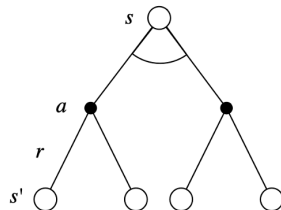
Real-time Dynamic Programming

- **Idea:** Focus on states **that are relevant to the agent** (we need an agent to interact with the world, to guide the priority over backup updates)
- Use agents experience to guide the selection of states
- After each time-step s_t, a_t, r_{t+1}
- Backup the state s

$$V(s_t) \leftarrow \max_a \left(r(s_t, a_t) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s_t, a_t) V(s') \right)$$

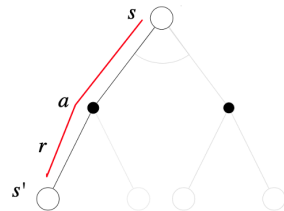
Full-Width Backups

- Standard DP uses full-width backups
- For each backup (sync or async)
 - Every successor state and action is considered
 - Using the true model of transitions and reward function
- DP is effective for medium-sized problems (millions of states)
- For large problems DP suffers from **curse of dimensionality**
 - Cost of backup grows exponentially with number of state variables
- Even one full backup can be too expensive



Sample Backups

- In subsequent lectures we will consider **sample backups**
- Using sample rewards and sample transitions $\langle s, a, r, s' \rangle$ (Instead of reward function r and transition dynamics $\mathbf{P}$)



Advantages:

- **Model-free**: no advance knowledge of MDP required
- Breaks the curse of dimensionality through sampling
- Cost of backup is constant, independent of $n = |S|$

- 1 Policy Evaluation
- 2 Policy Improvement
- 3 Policy Iteration and Value Iteration
- 4 Value Function in Theory

Background: Contraction Mapping Theorem

Definition: γ -contraction

An operator F on a normed vector space X is a γ -contraction, for $0 < \gamma < 1$ provided for all $x_1, x_2 \in X$:

$$\|F(x_1) - F(x_2)\| \leq \gamma \|x_1 - x_2\|$$

Theorem

***Contraction Mapping Theorem.** For any metric space X that is complete (i.e., closed) under an operator $F(x)$, where F is a γ -contraction,*

- *F converges to a unique fixed point in X*
- *At a linear convergence rate of γ*

Background: ∞ -Norm (Uniform/Maximum Norm)

We will measure distances between the state-value function u and v by the ∞ -norm, *i.e.*, the largest difference between state values,

$$\|u - v\|_{\infty} = \max_s |u(s) - v(s)|$$

$$\|u\|_{\infty} = \max_s |u(s)|$$

Value Function Space

- Consider the vector space V over value functions
- There are $|S|$ dimensions
- Each point in this space fully specifies a value function $V(s)$
- Bellman backup brings value functions closer in this space
- Therefore, the backup must converge to a unique solution

Bellman Expectation Backup in a Contraction

- Define the **Bellman expectation backup** operator

$$F_{\pi}(V) = \mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} V$$

- This operator is a γ -contraction, *i.e.*, it makes value functions closer by at least γ ,

$$\begin{aligned} \|F_{\pi}(U) - F_{\pi}(V)\|_{\infty} &= \|(\mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} U) - (\mathbf{r}_{\pi} + \gamma \mathbf{P}_{\pi} V)\|_{\infty} \\ &= \|\gamma \mathbf{P}_{\pi}(U - V)\|_{\infty} \\ &\leq \|\gamma \mathbf{P}_{\pi}(\mathbf{1} \|U - V\|_{\infty})\|_{\infty} \\ &= \|\gamma (\mathbf{P}_{\pi} \mathbf{1}) \|U - V\|_{\infty}\|_{\infty} \\ &= \|\gamma \mathbf{1} \|U - V\|_{\infty}\|_{\infty} \\ &= \gamma \|U - V\|_{\infty} \end{aligned}$$

Convergence of Iter. Policy Evaluation and Policy Iteration

- The Bellman expectation operator $\mathbf{P}_\pi$ has a unique fixed point
- V_π is a fixed point of $\mathbf{P}_\pi$ (by Bellman expectation equation)
- By the [contraction mapping theorem](#)
 - Iterative policy evaluation convergence on V_π
 - Policy iteration converges on V_*

Bellman Optimality Backup is a Contraction

- Define the **Bellman optimality backup** operator

$$F_*(V) = \max_{a \in A} r_a + \gamma P_* V$$

- This operation is a γ -contraction, *i.e.*, it makes value functions closer by at least γ (similar to previous proof)
- Convergence of Value Iteration:**
 - The Bellman optimality operator P_* has a unique fixed point
 - V_* is a fixed point of P_* (by Bellman optimality equation)
 - By the contraction mapping theorem, value iteration converges on V_*

Extra Reading Materials

- Chapter 4 (Dynamic Programming), Reinforcement Learning: An Introduction, Sutton & Barto.